



Centrul de Excelență în Informatică și Tehnologii Informaționale

Catedra Informatică I

Lucrare de studiu individual Nr.1

Disciplina: Asistență pentru programarea vizuală

Tema: Aplicație vizuală cu utilizarea controlerelor de bază

Realizat: Bruma Alexandrina, grupa P-2331

Verificat: Covali Eugenia

Nota _____

Conținut:

Introducere:	2
Aplicație vizuală cu utilizarea controlerelor de bază:	2
Plasarea componentelor pe suprafața ferestrelor:	2
Descrierea controllerelor:	3
Elementele caracteristice ale componentelor: proprietăți, metode, evenimente:	3
Aplicație vizuală complexă cu implementarea controlerelor frecvent utilizate:	4
Alte componente frecvent utilizate:	4
Adăugarea, excluderea și modificarea ferestrelor(MDI):	4
Conversia șirurilor de caractere:	5
Capturi de ecran cu aplicația creată:	6
Concluzie:	10
Bibliografie:	10

Introducere:

În ziua de astăzi, aplicațiile vizuale sunt găsite peste tot. Ele ne ajută să îndeplinim niște sarcini sau nevoi zilnice sau ocazionale. Limbajul de programare C# ne ajută să facem exact asta, iar combinat cu arhitectura .NET avem posibilitatea de a dezvolta aplicații bine structurate și comode de utilizat. La acest studiu individual, am creat o aplicație de tip Windows Form App care reprezintă o platformă de gestionare a unui magazin pentru animale, din punctul de vedere al unui utilizator. În aplicație am folosit nu doar cele mai folosite, dar și cele mai potrivite controale pentru acest proiect. Astfel, am creat pentru utilizatori o interfață comodă, intuitivă și prietenoasă.

Aplicație vizuală cu utilizarea controlerelor de bază:

Plasarea componentelor pe suprafața ferestrelor:

Plasarea componentelor în cadrul aplicației a fost realizată utilizând interfața grafică oferită de Visual Studio, prin metoda *drag and drop*. Ulterior, proprietățile fiecărui control au fost configurate din fereastra *Properties*, unde au fost modificate elemente precum numele (Name), culoarea (BackColor / ForeColor), fontul, dimensiunea și alte caracteristici vizuale.

Astfel, fiecare componentă a fost personalizată atât din punct de vedere funcțional (la nivel de cod), cât și din punct de vedere estetic, pentru a asigura coerență vizuală și o experiență plăcută pentru utilizator.

Descrierea controllerelor:

- **Label** - control utilizat pentru afișarea unui text informativ sau descriptiv în interfața aplicației.
- **Button** - control interactiv care execută o acțiune sau declanșează un eveniment atunci când este apăsat de utilizator.
- **DataGridView** - control folosit pentru afișarea și manipularea datelor sub formă de tabel, organizate pe rânduri și coloane.

- **DateTimePicker** - control care permite selectarea unei date și ore prin intermediul unei interfețe calendaristice sau a unui selector de timp.
- **ComboBox** - control care oferă utilizatorului posibilitatea de a selecta o opțiune dintr-o listă derulantă.
- **CheckBox** - control care permite selectarea sau deselectarea unei opțiuni, având două stări posibile (bifat/ne-bifat).
- **TextBox** - control utilizat pentru introducerea și editarea datelor text de către utilizator.

Elementele caracteristice ale componentelor: proprietăți, metode, evenimente:

1. Button:

Metode: Click(object sender, EventArgs e)

Proprietăți: BackColor, ForeColor, AutoSize, Text, TextAlign, TabIndex, Padding, Size

2. DataGridView:

Proprietăți: BackgroundColor, GridColor, RightToLeft, RowTemplate, TabIndex

3. CheckBox:

Metode: Checked

Proprietăți: Text, AutoCheck, Appearance (Normal/Button), TabIndex

4. ComboBox/TextBox:

Metode: Text / Value

Proprietăți: Text, CharacterCasing, MaxLength, Multiline, TabIndex

5. DateTimePicker:

Metode: Value, TimeOfDay

Proprietăți: Value, LeftToRightLayout, Format, MaxDate, MinDate, TabIndex

Aplicație vizuală complexă cu implementarea controlerelor frecvent utilizate:

Alte componente frecvent utilizate:

- **NumericUpDown** - permite introducerea unei valori numerice prin incrementare/decrementare.
- **RichTextBox** - similar cu **TextBox**, dar permite formatare avansată a textului.
- **Panel** - container folosit pentru gruparea altor controale.
- **FlowLayoutPanel** - aranjează automat controalele pe rând sau coloană.
- **PictureBox** - afișează imagini în interfață.
- **ListView** - afișează date sub formă de listă detaliată sau iconuri.
- **ErrorProvider** - afișează mesaje vizuale pentru validarea greșită a datelor.

Adăugarea, excluderea și modificarea ferestrelor(MDI):

Pentru implementarea structurii de tip MDI (Multiple Document Interface), a fost utilizată proprietatea **IsMdiContainer**, care se setează pe formularul principal și îl transformă în container părinte. Acesta devine spațiul în care sunt afișate toate ferestrele copil (MDI Child Forms).

Ferestrele copil sunt create și afișate în urma unor evenimente specifice, cum ar fi apăsarea unui buton. La apelare, fiecărei forme i se setează proprietatea **MdiParent**, indicând formularul principal ca părinte, după care aceasta este afișată folosind metoda **Show()**.

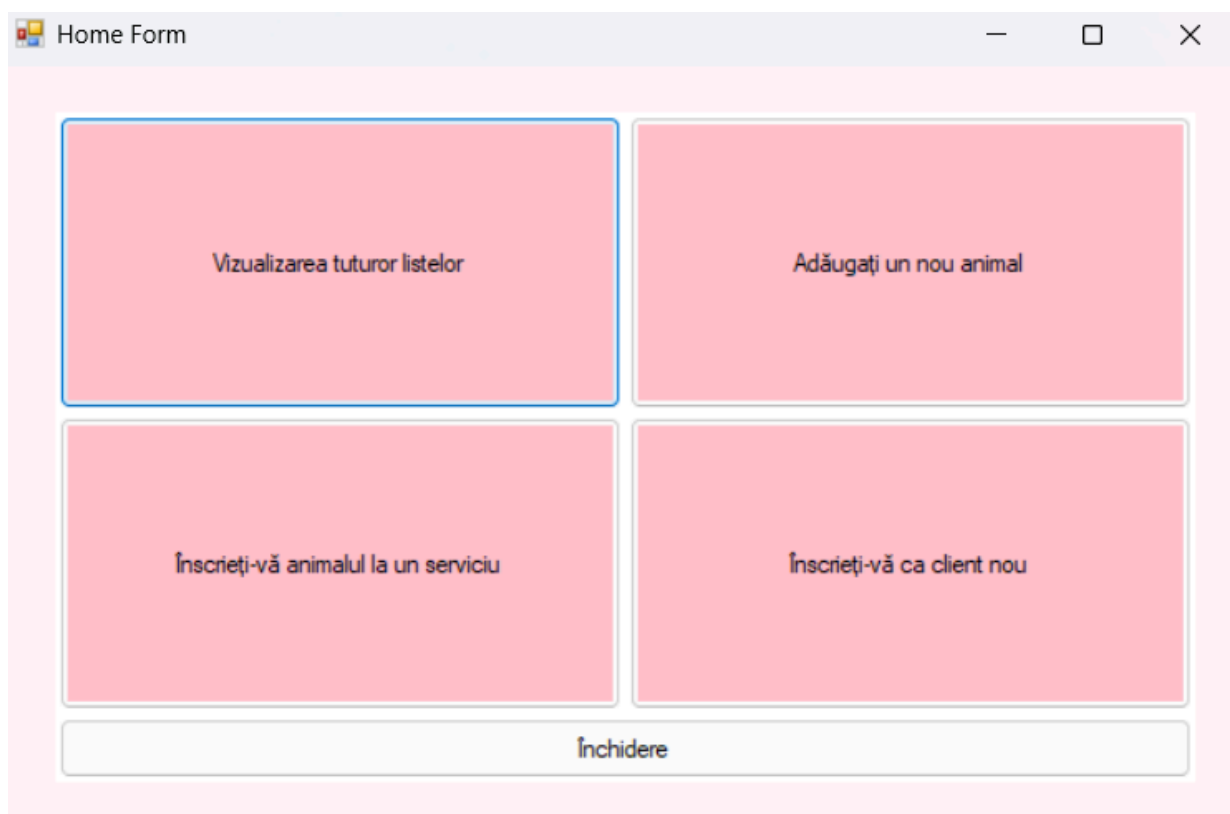
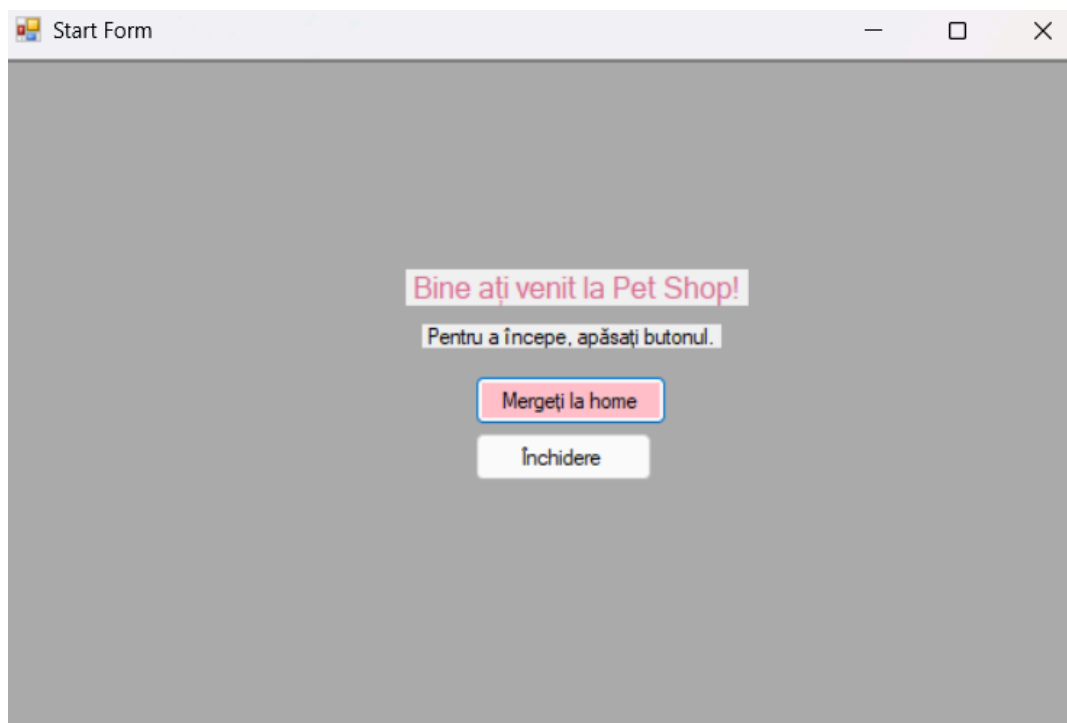
În cadrul aplicației dezvoltate, utilizarea ferestrelor MDI a contribuit la organizarea clară a funcționalităților și la structurarea eficientă a interfeței, oferind utilizatorului o experiență intuitivă și ușor de navigat.

Conversia șirurilor de caractere:

În cadrul aplicației, a fost necesară conversia șirurilor de caractere introduse de utilizator în valori numerice. Datele au fost preluate din componente de tip `TextBox`, care stochează informațiile sub formă de text. Pentru transformarea valorilor de tip șir de caractere în valori întregi (`int`), a fost utilizată metoda `int.Parse()`. Exemplu:

```
IdStapan = int.Parse(ownerTextBox.Text);
```

Capturi de ecran cu aplicația creată:



Alegeți lista căutată

Lista animalelor

Lista clienților

Lista serviciilor

Lista taberelor de zi

Lista tuturor magazinelor

Înapoi la home

	Nume	dataNasterii	Specie
▶	Bella	20.11.2019	Pisica
	Thomas	02.11.2020	Pisica
	Daisy	20.07.2022	Caine
	Penny	01.11.2023	Iepure
	Fluffy	06.04.2017	Caine

Opțiunea 1: Vizualizarea tuturor listelor -> Lista Animalelor

Animal Form

INTRODUCETI DATELE ANIMALULUI DE COMPANIE:

Aria

28.02.2024

Cal

3

Animalul a fost adaugat!

Adaugă animalul!

Înapoi la home

Opțiunea 2: Adăugați un nou animal -> Adaugă animalul

Appointment Form

INTRODUCETI DATELE REZERVĂRII:

2 15:00:00

3 16:00:00

2

Rezervarea a fost făcută cu succes!

```
Button clicked
Rezervare facuta:
Serviciul: 2, animalul: 3, clientul: 2, intre orele: 15:00:00 - 16:00:00
```

Opțiunea 3: Înscrieți-vă animalul la un serviciu -> Înscrie!

Camp Form

INTRODUCETI DATELE REZERVĂRII:

2 15:00:00

1 09:00:00

marți , 3 martie 20: ☒ Ședere peste noapte

Rezervarea a fost făcută cu succes!

```
Rezervare facuta:
Animalul: 2, tabara: 1, data: 03.03.2026 21:10:09, intre orele: 15:00:00 - 09:00:00, cu sedere pe noapte: True
```

Opțiunea 3: Înscrieți-vă animalul la un serviciu -> Înscrieți-vă la tabăra de zi

Client Form

ÎNREGISTRAȚI-VĂ

Bruma Alexandrina

067889955 alexandrinabruma@gmail.com

str.Sarmizegetusa 48

Înregistrarea a fost făcută cu succes!

Înregistrare! Înapoi la home

Inregistrare facuta:
Client: Bruma Alexandrina, telefon: 067889955, email: alexandrinabruma@gmail.com, adresa: str.Sarmizegetusa 48

Opțiunea 4: Înscrieți-vă ca client nou -> Înregistrare!

Concluzie:

În concluzie, acest studiu individual m-a ajutat să îmi dezvolt deprinderile de lucru cu limbajul de programare C#, precum și competențele necesare pentru crearea unei aplicații vizuale. Fiind o direcție de programare complet nouă pentru mine, am evoluat semnificativ, învățând concepte precum utilizarea listelor și a clasei DataStore, realizarea binding-ului în DataGridView, definirea proprietăților prin { `get`; `set`; } și implementarea validărilor pentru datele introduse.

Un aspect esențial al acestui proiect a fost înțelegerea modului de construire a unei aplicații bazate pe o bază de date și organizarea unei structuri pregătite pentru conectarea ulterioară la aceasta. Mi s-a părut deosebit de interesant procesul de dezvoltare a interfeței, urmărind transformarea ideii inițiale, de la schema bazei de date, într-o aplicație complet funcțională, cu metode, opțiuni pentru utilizator și o interfață adaptată temei alese.

Pe parcurs, am întâmpinat și dificultăți, în special în gestionarea conversiilor între tipuri diferite de date și în proiectarea structurii claselor în funcție de relațiile existente în baza de date. Aceste provocări m-au ajutat însă să înțeleg mai bine logica aplicațiilor orientate pe obiect și importanța unei arhitecturi corecte.

În final, am reușit să dezvolt o aplicație coerentă, logic structurată și accesibilă utilizatorului, oferind o soluție utilă pentru viața de zi cu zi a stăpânilor de animale.

Bibliografie:

- [Tutorial GeeksforGeeks C#](#)
- [Tutorial Microsoft](#)
- [Tutorial Multiple-Document Interface \(MDI\) Applications](#)
- [ChatGPT](#)